Analysis of Some Search Algorithms on Hexagonal Maze Problem

Sahil Raj
Indian Institute of Technology Patna

February 20, 2025

Contents

1 Introduction **2**

2 Maze Generation Using Iterative Randomized Kruskal's Algorithm **3**
2.1 Overview of Kruskal's Algorithm 3
2.2 Iterative Randomized Kruskal's Algorithm for Hexagonal Mazes 3
2.3 Examples 4

3 Search Algorithms for Hexagonal Maze Navigation **5**
3.1 Depth-First Search (DFS) 5
3.2 Breadth-First Search (BFS) 6
3.3 A^* Search 7
3.4 Beam Search 8
3.5 Analysis of Results 9
3.6 Discussion 10

A Formulation of Hexagonal Maze **11**

B Formulation of Hexagonal Maze into a Search Problem **14**

1
Introduction

Pathfinding in mazes is a fundamental problem in artificial intelligence and graph search. While rectangular mazes conform naturally to the Cartesian coordinate system, hexagonal mazes present unique structural and computational challenges due to their six-directional movement and modular coordinate representation.

In this paper, We formally define a *hexagonal maze* and formulate it as a *search problem*. To generate solvable mazes, we use a modified **Iterative Randomized Kruskal's algorithm**, ensuring that each maze has a single unique solution. We then apply four search algorithms—*Depth-First Search* (DFS), *Breadth-First Search* (BFS), *A* Search*, and *Beam Search*—to analyze their computational performance.

We compare these algorithms based on execution time and memory usage across different maze sizes. The results show that **BFS** has *high memory consumption* due to its exhaustive search, while **DFS** is more *memory-efficient* but can explore deeper paths unnecessarily. **A*** performs efficiently when guided by a well-chosen heuristic, whereas **Beam Search** reduces memory usage by restricting exploration breadth. These findings highlight the trade-offs between computational efficiency and resource constraints in solving hexagonal mazes.

This study provides insights into selecting appropriate search strategies for hexagonal grid traversal, which has applications in robotics, game AI, and network routing. Future work could explore heuristic optimizations for A*, adaptive beam widths for Beam Search, and applications in dynamic and weighted mazes.

2

Maze Generation Using Iterative Randomized Kruskal's Algorithm

The generation of a solvable hexagonal maze follows an **iterative randomized Kruskal's algorithm**, which ensures that the resulting maze is a spanning tree—a connected graph with no cycles. This guarantees that there is a unique solution for every pair of start and goal cells.

2.1 Overview of Kruskal's Algorithm

Kruskal's algorithm is a classical algorithm used to construct *minimum spanning trees* (**MSTs**) from an undirected graph. The standard version operates as follows:

1. **Initialize:** Treat each node (cell) as a separate connected component.
2. **Sort Edges:** Sort all possible edges (walls) of the graph in random order.
3. **Union-Find:** Iterate through the edges, adding an edge to the maze if it connects two previously disconnected components. This prevents cycles from forming.
4. **Repeat** until a single connected component remains, forming a spanning tree.

For hexagonal mazes, the algorithm is adapted to work with the maze described in **Appendix A**.

2.2 Iterative Randomized Kruskal's Algorithm for Hexagonal Mazes

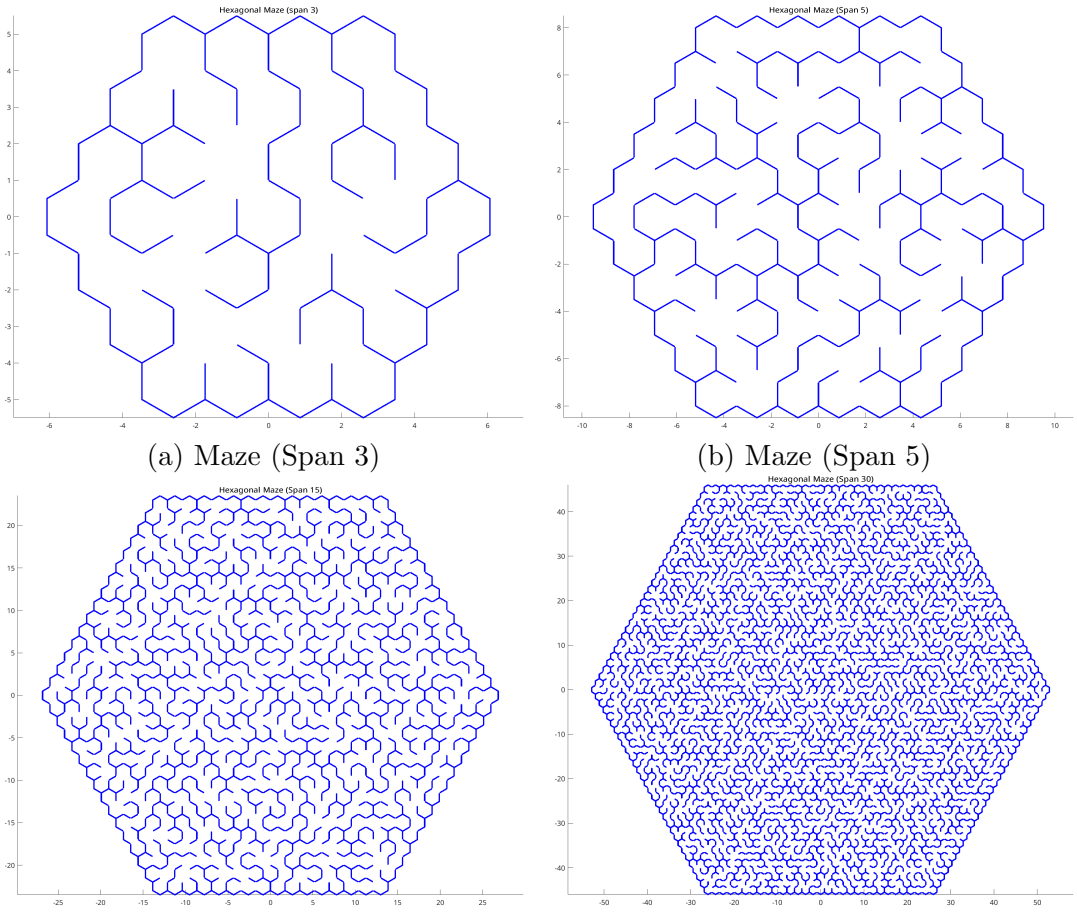
To generate a hexagonal maze using iterative randomized Kruskal's algorithm, the following approach is used:

1. **Initialize:** The maze consists of *cells* arranged in a hexagonal tessellation of span i. Each cell starts as an isolated component in a *disjoint-set data structure (Union-Find)* and a list of *all possible walls (edges)* in the maze is created.
2. **Randomized Edge Processing:** The list of walls is shuffled randomly to introduce randomness in maze generation. Iteratively, each wall is considered, if the two adjacent cells separated by the wall belong to different components, the wall is removed, and the components are merged else the wall remains to prevent cycles.
3. **Termination Condition:** The process continues until *all cells belong to a single connected component*, ensuring a fully connected, acyclic hexagonal maze with a unique solution.

The properties of the Generated Maze are:

1. **Guaranteed Solvability:** Since the maze is a spanning tree, every cell is reachable from any other cell.
2. **Unique Solution:** There is exactly one path from the start cell σ to the goal cell γ, making shortest-path comparisons irrelevant.
3. **Uniform Randomness:** The randomness of wall selection ensures diverse maze structures while maintaining solvability.

2.3 Examples



(a) Maze (Span 3) (b) Maze (Span 5)

(c) Maze (Span 15) (d) Maze (Span 30)

Figure 2.1: Hexagonal Maze Generation of Different Spans

Any two distinct *cells* can be chosen as *start cell* and *goal cell*, which completes the maze-formation process.

3

Search Algorithms for Hexagonal Maze Navigation

In this section, we discuss and compare various search algorithms used to navigate the generated hexagonal mazes. Since the maze is a spanning tree, it contains a unique path from the *start cell* to the *goal cell*. The algorithms analyzed include *Depth-First Search* (DFS), *Breadth-First Search* (BFS), *A* Search*, and *Beam Search*. Each algorithm is implemented and evaluated based on key performance metrics such as the number of nodes explored, memory usage, and execution efficiency.

Note: The mazes for following section has same span, but the mazes are different.

3.1 Depth-First Search (DFS)

DFS is a fundamental uninformed search algorithm that explores a path deeply before backtracking. It operates using a *Last-In-First-Out (LIFO) stack*, meaning it prioritizes depth over breadth.

Algorithm Workflow

1. Start from the initial cell, marking it as visited.
2. Push it onto a stack.
3. Iteratively explore the last-added cell's neighbors, adding unvisited ones to the stack.
4. If the goal is reached, the algorithm terminates. Otherwise, backtracking occurs.

Performance Analysis

- **Time Complexity:** $O(n)$ (since every node is visited once).
- **Space Complexity:** $O(n)$ in the worst case (deep recursion).
- **Behavior in the Hexagonal Maze:** DFS occasionally explore long, inefficient paths before finding the goal. However, since the maze has a single unique solution, DFS will always reach the goal given sufficient recursion depth.

Solution of Maze with DFS

IIterations	248
Nodes Explored	248
Path Length	21
Visited Nodes Stored	331
Parent Map Size	256
Max Stack Size	15

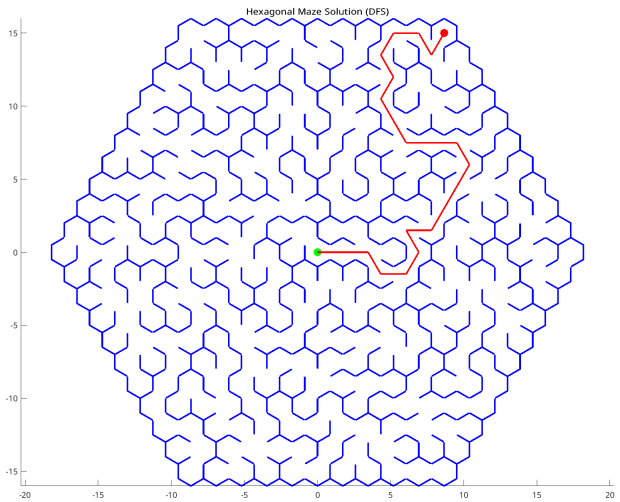


Figure: Solution of **Span-10 Hexagonal Maze** with center as *start cell* and top-right as *goal cell* with *DFS*

3.2 Breadth-First Search (BFS)

BFS is another uninformed search technique that explores the maze layer by layer using a *First-In-First-Out (FIFO)* queue. It guarantees finding the shortest path in an unweighted graph.

Algorithm Workflow

1. Begin at the start cell and enqueue it.
2. Dequeue a cell, mark it as visited, and enqueue all unvisited neighbors.
3. Continue until the goal is reached.

Performance Analysis

- **Time Complexity**: $O(n)$, as each cell is visited once.
- **Space Complexity**: $O(n)$, since BFS must store all frontier nodes.
- **Behavior in the Hexagonal Maze**: Since the maze has a single solution, BFS efficiently finds it. It is particularly useful in scenarios where shortest paths are desirable.

Solution of Maze with BFS

Iterations	226
Nodes Explored	226
Path Length	23
Visited Nodes Stored	331
Parent Map Size	234
Max Queue Size	27

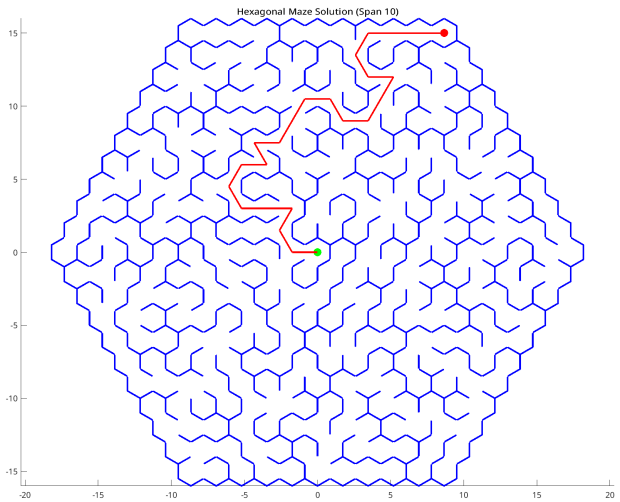


Figure: Solution of **Span-10 Hexagonal Maze** with center as *start cell* and top-right as *goal cell* with *BFS*

3.3 A^* Search

A^* is an informed search algorithm that combines BFS-like exploration with a heuristic function, making it more efficient in finding paths. It maintains two cost values:

- $g(n)$: Cost from the start node to the current node.
- $h(n)$: Heuristic cost estimating the remaining distance to the goal.

A^* prioritizes nodes based on $f(n) = g(n)+h(n)f(n) = g(n)+h(n)$, where $h(n)$ is computed using *euclidean distance* in the hexagonal grid.

Algorithm Workflow

1. Start from the initial cell and add it to a priority queue.
2. Expand the node with the lowest $f(n)$.
3. Update the cost of neighbors and continue until the goal is reached.

Performance Analysis

1. **Time Complexity:** $O(nlogn)$ due to priority queue operations.
2. **Space Complexity:** $O(n)$.
3. **Behavior in the Hexagonal Maze:** A^* efficiently finds the solution while expanding fewer nodes than BFS. It is ideal when heuristic-based guidance is beneficial.

Solution of Maze with A^*

Iterations	27
Nodes Explored	27
Path Length	19
gScore Map Size	331
fScore Map Size	331
Parent Map Size	38
Max Open Set Size	12

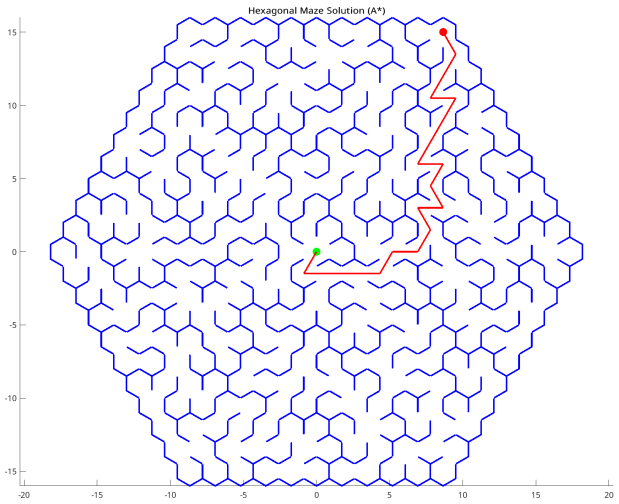


Figure: Solution of **Span-10 Hexagonal Maze** with center as *start cell* and top-right as *goal cell* with A^*

3.4 Beam Search

Beam Search is a memory-efficient variation of BFS that limits the number of nodes explored at each level based on a beam width k. Instead of expanding all possible paths, it only keeps the best k candidates based on the heuristic.

Algorithm Workflow

1. Start at the initial cell and maintain a list of up to k best candidates per iteration.
2. Expand only the top k nodes based on heuristic scores.
3. Continue until the goal is reached or no nodes remain.

Performance Analysis

- **Time Complexity:** $O(kn)$ (dependent on beam width).
- **Space Complexity:** $O(k)$ (better than BFS).
- **Behavior in the Hexagonal Maze:** Beam search is effective in reducing memory usage but may fail if the beam width is too small, leading to incomplete solutions.

Solution of Maze with Beam Search

Iterations	21
Nodes Explored	58
Path Length	21
Parent Map Size	71
Max Level Size	5

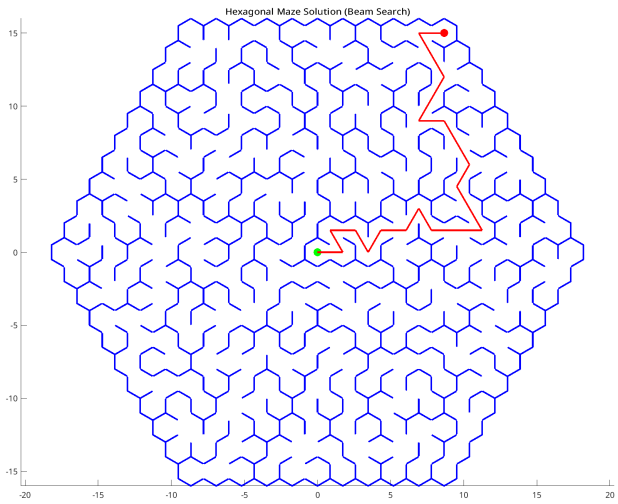


Figure: Solution of **Span-10 Hexagonal Maze** with center as *start cell* and top-right as *goal cell* with *Beam Search*.

3.5 Analysis of Results

To evaluate the performance of each search algorithm, we measured key metrics such as iterations, nodes explored, path length, memory usage, and the maximum frontier size. The results are summarized in Table 3.1.

Algorithm	Iterations	Nodes Explored	Memory Usage	Max Frontier Size
DFS	248	248	587	15
BFS	226	226	565	27
A*	27	27	700	12
Beam Search	21	58	76	5

Table 3.1: Comparison of search algorithms based on performance metrics

Additionally, here are the results when all these algorithms were run for different values of *span*(*The performance comparison are on identical mazes*).

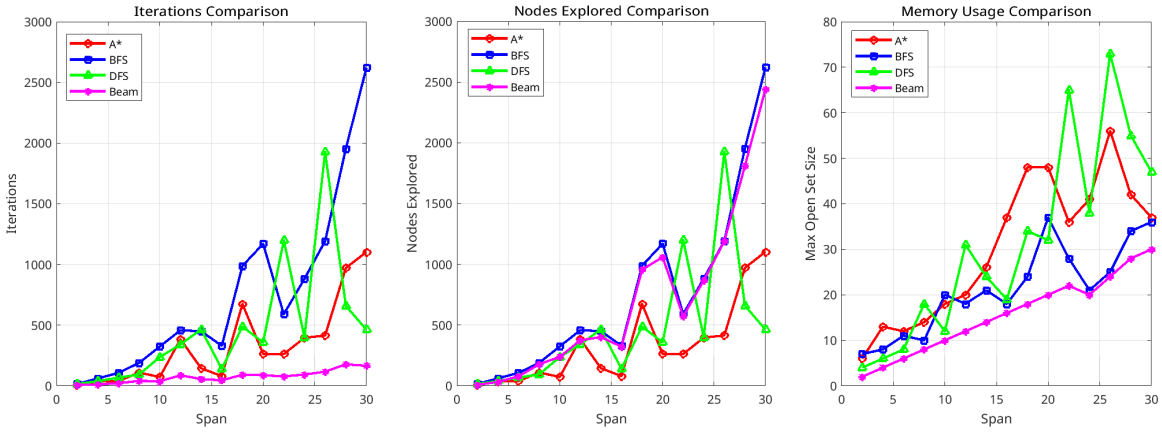


Figure: The comparison of algorithms performance stats(k value for *Beam Search* is kept equal to *span* of the maze).

The performance of each algorithm reflects its underlying search strategy and the trade-offs involved in solving a structured maze. Given that the maze has a unique solution due to its spanning tree nature, the evaluation focuses on efficiency, memory consumption, and the overall effectiveness of each approach.

Depth-First Search (DFS) follows an aggressive exploration strategy, diving deep into a path before backtracking when necessary. This approach results in extensive node exploration, as it does not prioritize efficient traversal. However, it benefits from low memory usage due to its stack-based implementation. While it guarantees a solution, its lack of path optimization makes it an exhaustive but often inefficient choice.

Breadth-First Search (BFS), on the other hand, systematically explores all possible paths at each level before proceeding deeper. As a result, BFS requires significantly more memory, as it maintains multiple active paths simultaneously. While it is more structured and effective in unweighted search spaces, its high memory footprint can be a limiting factor in complex mazes.

A Search* demonstrates the advantages of heuristic-driven exploration. By using an estimation of the remaining distance to the goal, it reduces unnecessary expansions and focuses on the most promising paths. This results in a significantly smaller number of explored nodes while maintaining a balance between optimality and efficiency. Although it requires additional memory to store heuristic values, its overall computational efficiency makes it a superior choice for navigating structured mazes.

Beam Search operates within a constrained search space by limiting the number of active paths at each step. While this leads to reduced memory usage, it comes at the cost of increased node exploration, as the algorithm may discard potentially optimal paths. Although it performs well in terms of iterations, its restrictive nature prevents it from consistently finding the solution. The trade-off between efficiency and completeness makes it a more unpredictable approach compared to A*.

Each algorithm exhibits distinct strengths and weaknesses. DFS prioritizes depth over efficiency, BFS ensures the shortest path at the cost of memory, A* achieves an optimal balance through informed search, and Beam Search sacrifices completeness for controlled resource usage.

3.6 Discussion

These observations highlight the fundamental role of search strategies in structured problem-solving. In a hexagonal maze with a unique solution, the challenge is not to find an optimal path among many but to *efficiently discover the only existing route*. Algorithms like *BFS* and *A** ensure systematic exploration, with A* leveraging heuristic guidance to minimize unnecessary expansions. *DFS*, despite its depth-first approach, reliably reaches the solution, though its order of exploration may vary. *Beam Search*, constrained by its limited breadth, may discard potentially useful paths early, affecting its efficiency in certain cases.

Ultimately, when the solution is *singular*, the choice of an algorithm is dictated by computational efficiency rather than path quality. The objective is to traverse the structured space with minimal resource consumption while ensuring a guaranteed resolution.

A
Formulation of Hexagonal Maze

This section is intended to give formal definitions for *Hexagonal Maze*. It resembles the commonly known structure *honeycomb*.

The ease of definition of *Rectangular Maze* owing to the good conformity with Cartesian system of coordinates leaves us when we deal with hexagonal structures. However, the *honeycomb* structure is highly symmetric and a simpler alternative way can be developed to represent structure.

The fundamental object of the whole structure is a regular hexagon, which repeats itself as miniature. From now on, this object will be called *cell*. These *cells* combine together to form *hexagonal tessellation*.

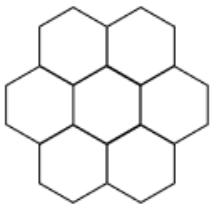


Figure 1: The *hexagonal tessellation* with seven hexagons.

Instead of Cartesian coordinate system, we are going to use a modular one. The formation of the structure follows a sequence.

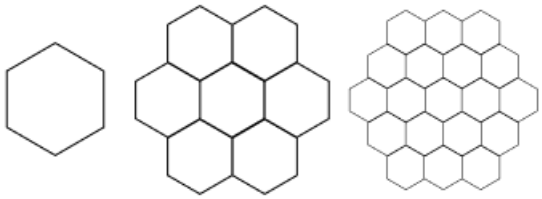
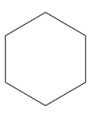
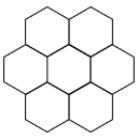
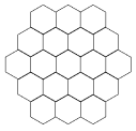
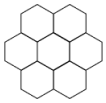
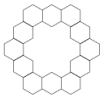


Figure 2: The first, second and third structure in the sequence.

Let $\{S_n\}$ denote that sequence with S_0 being just a regular hexagon. Each subsequent terms of the sequence of the sequence is formed by addition of some set of hexagons.
For example:

For $S_0 =$  , $S_1 =$  and $S_2 =$ 

Let $\mathcal{B}_1 =$  and $\mathcal{B}_2 =$  .

Then the formation can be represented by addition of structures. $S_0 + \mathcal{B}_1 = S_1$ and $S_1 + \mathcal{B}_2 = S_2$. In general form: $S_{i-1} + B_i = S_i$ for $i \geq 1$.

Those sets $\mathcal{B}_i$ we will call *perimeter*, where i is *level of perimeter*. One can easily observe that $|\mathcal{B}_0| = 1$, for it is just a regular hexagon. $|\mathcal{B}_1| = 6$ and $|\mathcal{B}_2| = 12$. The term $|\mathcal{B}_i|$ progress like an arithmetic progression with $|\mathcal{B}_i| = 6i$.

Note: For labeling, the orientation of boundary is assumed as is given for $\mathcal{B}_1$ and $\mathcal{B}_2$

For each *perimeter* $\mathcal{B}_i$, we will index its element ranging from $[0, 6i)$. The *top-left* hexagon is labeled as 0 and then incrementally clockwise.

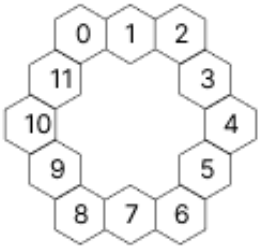


Figure 3: The labeling of $\mathcal{B}_2$ perimeter.

Following this scheme, each *cell* can be uniquely identified by two numbers, i.e., the *level of perimeter* and *label* in the perimeter.
Thus a *cell* can be represented as a *2-tuple* $c = (l, m)$ where l is the *level of perimeter* and m is its *label* in the perimeter. Using this definition of the *cell*, we can define *perimeter* and *hexagonal tessellation* S_i.

$$\mathcal{B}_i := \{(i, j) | j \in [0, 6i)\}$$
$$S_i = \bigcup_{j=0}^{i} \mathcal{B}_j$$

For a *hexagonal tessellation* S_i, we will call i as its *span* and B_i is called it's *boundary*.

Let's assume a *cell* $c : (l, m)$. The neighbors of c can have *level of perimeter* as $l-1, l$ or $l+1$. However for $l = 0$, it has six neighbors with *level of perimeter* valued 1.

$$neib : (0, 0) \to \{(1, 0), (1, 1), (1, 2), (1, 3), (1, 4), (1, 5)\}$$

For $l >= 1$, let $w = \lfloor \frac{m}{l} \rfloor$ and $i = m\%l$
If $m \equiv 0 \bmod l$, then it has 2, 1, and 3 neighbors having *level of perimeter* as $l, l-1$ and $l+1$ respectively provided $l < span\{S_i\}$, otherwise $l+1$ corresponds to 0 neighbors.

So, if $l < span\{S_i\}$, then

$$neib : (l, m) \to \{(l, wl - 1), (l, wl + 1)\}\%6l \cup$$
$$\{l - 1, w(l - 1)\}\%6(l - 1) \cup$$
$$\{(l + 1, w(l + 1)), (l + 1, w(l + 1) - 1), (l + 1, w(l + 1) + 1)\}\%6(l + 1)$$

or else

$$neib : (l, m) \to \{(l, wl - 1), (l, wl + 1)\}\%6l \cup$$
$$\{l - 1, w(l - 1)\}\%6(l - 1)$$

Now, if $m \not\equiv 0 \bmod l$, then it has 2 neighbors from each *level of perimeter* $l, l-1$ and $l+1$ provided $l < span\{S_i\}$, otherwise $l+1$ corresponds to 0 neighbors.

Again, if $l < span\{S_i\}$, then

$$neib : (l, m) \to \{(l + 1, w(l + 1) + i), (l + 1, w(l + 1) + i + 1)\}\%6(l + 1) \cup$$
$$\{(l - 1, w(l - 1) + i - 1), (l - 1, w(l - 1) + i)\}\%6(l - 1) \cup$$
$$\{(l, wl + i - 1), (l, wl + i + 1)\}\%6l$$

or else

$$neib : (l, m) \to \{(l-1, w(l-1)+i-1), (l-1, w(l-1)+i)\}\%6(l-1) \cup \{(l, wl+i-1), (l, wl+i+1)\}\%6l$$

For each pair of *cells* $c : (l, m)$ and $c' : (l', m')$ such that $c \neq c'$. If c, c' are neighbors, then the *4-tuple* $w : (c, c')$ is defined as *inner-wall* between cells c and c'. The set of all *inner-walls* of *hexagonal tessellation* is denoted by $\mathcal{W}$.

Now that all essential elements are defined, we define the **Hexagonal Maze**($\mathcal{H}$) as a *5-tuple*

$$\mathcal{H} := (S, \Omega, \mu, \sigma, \gamma)$$

S is the set of all *cells*
$\Omega \subseteq \mathcal{W}$ is the set of *inner-walls*
$\mu : S \to 2^S$ is the function that returns set of neighbors of a *cell*
$\sigma \in S$ is the start cell
$\gamma \in S$ is the goal cell

We say that two *cells* c and c' are *connected* if they are neighbors satisfying $(c, c') \notin \mathcal{W} \wedge (c', c) \notin \mathcal{W}$.

The *cell* c' is called reachable from *cell* c if there exist a finite sequence of *cells* $c_1, c_2, ..., c_k$, $c_1 = c$ and $c_k = c'$ such that c_i and c_{i+1} is *connected* for $1 \leq i \leq k-1$.

A *Hexagonal maze($\mathcal{H}$)* is called *solvable* if *cell* γ is *reachable* from *cell* σ.

B

Formulation of Hexagonal Maze into a Search Problem

This section will use concepts and definition from **Appendix A**. The formulation of a scenario into a search problem involves definition of *state space*, *start state*, *goal state*, *actions*, *transition function*.

For a **Hexagonal Maze($\mathcal{H}$)**, The *state space* consists of all possible *cells* in the maze. Consider each *cell* of S represented by (l, m) as a state, since in a maze, the *agent* can **only** be at any one *cell* at a particular time. Thus S represents the *state space*.

The problem defines *start state* $\sigma = (l_s, m_s) \in S$. This is the initial position where the search starts. Similarly the *goal state* is defined $\gamma = (l_g, m_g) \in S$.

From any *state*, that is a *cell* of $\mathcal{H}$, a *agent* can move to its *neighbors*. Thus an *action* is a map $\mu : S \to 2^S$, that returns the set of all accessible states from the current state.

A move from *state* c to c' is valid if and only if

$$(c, c') \notin \Omega$$

Meaning, an *agent* can make a move $c \to c'$, if there is no wall between c and c'. Let this be considered as the *transition model*.

The *state space* can be represented as a *graph*, where each *node* corresponds to a valid *state*. And there exist an *edge* between those pairs of nodes, which are not separated by *wall*. The type of *graph* formed by this *state space* is an *undirected* and *unweighted graph*.

There is a *solution* to this *search problem*, if the *cell* represented by the *goal state* is *reachable* from the *cell* represented by *start state* in the **Hexagonal Maze($\mathcal{H}$)**.

Thus the defined terms *states, goal state, start state, actions* and *transition model* together form the *search problem*. Its easy to see that the *search* problem conforms with the definition of *Hexagonal Maze* structurally.

On the graphical form of the problem, algorithms like *BFS, DFS, A*, BS* can be used to find the solution.

Bibliography

[1] Wikipedia contributors, "Maze generation algorithm," Wikipedia, The Free Encyclopedia. Available:`https://en.wikipedia.org/wiki/Maze_generation_algorithm`. [Accessed: Feb. 20, 2025].

[2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed., Pearson, 2016.